

WEEK - 4 MODULE

Algorithm Analysis

Algorithm Analysis is the process of evaluating an algorithm to determine how efficiently it uses time and memory as the size of the input increases.

The main objective of algorithm analysis are —

- Time complexity - measures the amount of time an algorithm takes (e.g., $O(1)$, $O(n)$, $O(n^2)$ etc)
- Space Complexity - measures the amount of memory an algorithm requires during execution.

Time Complexity - It measures how efficient an algorithm is in terms of execution time and memory usage.

Space Complexity - It measures the amount of extra memory required by an algorithm.

Asymptotic Notation - Asymptotic Notation is a mathematical method used to describe the time or space complexity of an algorithm as the input size becomes very large.

- Big $O()$ - upper bound (worst case)
- Big $\Omega()$ lower bound (best case)
- Big $\Theta()$ - Tight bound (average / exact growth).

Best, Average and Worst case -

Best case - The minimum time an algorithm takes to execute for the most favourable input.

e.g., linear search finds the element at the first position. ($O(1)$)

Average case - The expected time an algorithm takes for a typical or random input.

e.g., linear search find an element in the middle. - ($O(n)$)

Worst case - The maximum time an algorithm takes for the least favourable input.

e.g., linear search finds the element at the last position or not at all. ($O(n)$).

Rules to Calculate Complexity -

- One loop running N times - ($O(n)$)
- Two nested loops - ($O(n^2)$)
- Logarithmic loop (dividing by 2 each time) - ($O(\log n)$)
- Sequential loop - add complexities and keep the highest order.

Time Complexity of loops -

→ for ($i=0$; $i < n$; $i++$) → $O(n)$

→ for ($i=1$; $i \leq n$; $i* = 2$) → $O(\log n)$

Time Complexity of Nested loops -

```
for (i=0; i < n; i++)  
    for (j=0; j < n; j++)
```

Time complexity :- $O(n^2)$

Time Complexity of Logarithmic loops

```
for (i=1; i < n; i* = 2)
```

Time complexity $O(\log n)$

Space Complexity basics

- No extra memory ($O(1)$)
- Array of size N → $O(n)$
- matrix of size $n \times n = O(n^2)$.

Problem - Statement - 1

Given a positive integer N , print all the numbers from 1 to N in increasing order, separated by a single space.

e.g.,

Input

5

Output

1 2 3 4 5

Code

```
#include <iostream>
using namespace std;
int main ( ) {
```

```
    int N;
```

```
    cin >> N;
```

```
    for (int i = 1; i <= N; i++) {
```

```
        cout << i << " ";
```

```
    }
```

```
    return 0;
```

```
}
```

Time Complexity - $O(n)$

Space Complexity - $O(1)$

Explanation - The loop runs from 1 to N ; printing each number exactly once. Since the number of iterations depend on N , the time complexity is $O(n)$.

```
#include <iostream>
using namespace std;
```

```
✓ int main() {
    int N;
    cin >> N;

    ✓ for (int i = 1; i <= N; i++) {
        |     cout << i << " ";
        |     }

    return 0;
}
```

```
PS C:\Users\pragy\OneDrive\Desktop\week module4> cd "c:\Users\pragy\OneDrive\Desktop\week module4\" ; if ($?) { g++ p1.cpp -o p1 } ; if ($?) { .\p1 }  
5  
1 2 3 4 5  
PS C:\Users\pragy\OneDrive\Desktop\week module4> █
```


Problem Statement-2

Given a positive Integer N , find the sum of first N natural number using a loop.

e.g.,

Input

10

Output

55

Code

```
#include <iostream>
using namespace std;
int main () {
    int N;
    cin >> N;
    long long sum = 0;
    for (int i = 1 ; i <= N; i++) {
        sum += i;
    }
    cout << sum;
    return 0;
}
```


Time Complexity - $O(N)$

Space Complexity - $O(1)$

Explanation - The loop executes N times and adds each number to the sum. Therefore the running time increases linearly with N , giving $O(N)$ time complexity. Only one variable is used to store the sum, so space complexity is $O(1)$.

Optimized approach

Use the mathematical formula:

$$\text{Sum} = \frac{N(N+1)}{2}$$

This reduces the time complexity to $O(1)$


```
#include <iostream>
using namespace std;
```

```
✓ int main() {
    int N;
    cin >> N;

    long long sum = 0;

    ✓ for (int i = 1; i <= N; i++) {
        |     sum += i;
        |
    }

    cout << sum;

    return 0;
}
```



```
> cd "c:\Users\pragy\OneDrive\Desktop\week module4\" ; if ($?) { g++ p2.cpp -o p2 } ; if ($?) { .\p2 }  
10  
55  
PS C:\Users\pragy\OneDrive\Desktop\week module4> █
```

Problem Statement - 3

Given two integers A and B, calculate the value of $A \leq \sup > B \leq / \sup > (A^B)$ using a loop.

e.g.,

Input

2 5

Output

32

Code

```
#include <iostream>
```

```
using namespace std;
```

```
int main () {
```

```
    int A, B;
```

```
    cin >> A >> B;
```

```
    long long result = 1;
```

```
    for (int i = 1; i <= B; i++) {
```

```
        result * = A;
```

```
    }
```

```
    cout << result;
```

```
    return 0;
```

3

Time Complexity - $O(B)$

Space Complexity - $O(1)$

Explanation -

The loop multiplies A by itself B times. Hence, the running time depends on the value of B ; resulting in $O(B)$ time complexity. Only one extra variable is used, so the space complexity is $O(1)$.

Optimized Approach

A faster method is Binary Exponentiation (fast power), which computes the result in $O(\log B)$ time.

```
#include <iostream>
using namespace std;
```

```
✓ int main() {
    int A, B;
    cin >> A >> B;

    long long result = 1;

    ✓ for (int i = 1; i <= B; i++) {
        |     result *= A;
        | }

    cout << result;

    return 0;
}
```



```
PS C:\Users\pragy\OneDrive\Desktop\week module4> cd "c:\Users\pragy\OneDrive\Desktop\week module4\" ; if ($?) { g++ p3.cpp -o p3 } ; if ($?) { .\p3 }
2 5
32
PS C:\Users\pragy\OneDrive\Desktop\week module4> █
```